

SESSION 09 ASSIGNMENT

C# Advanced Topics & Unit Testing

Name: Ahmed Mohamed Saeed Nofal

Group: ONL5_SW5_S4

Track: Full Stack Web Development (.NET)

Part 1: Primary Constructor & Records

Q1. Patient Class with Primary Constructor & ToString Override

```
public class Patient(int id, string fullName, string phoneNumber, string medicalHistory)
{
    public int Id { get; } = id;
    public string FullName { get; } = fullName;
    public string PhoneNumber { get; } = phoneNumber;
    public string MedicalHistory { get; } = medicalHistory;

    public override string ToString()
    {
        return $"Id: {Id} :: FullName: {FullName} :: PhoneNumber: {PhoneNumber} ::
MedicalHistory: {MedicalHistory}";
    }
}
```

Q2. Testing Patient Class Instances (Equality & GetHashCode)

```
// Inside Main method:
Patient p1 = new(101, "Ahmed Nofal", "01012345678", "No chronic conditions");
Patient p2 = new(101, "Ahmed Nofal", "01012345678", "No chronic conditions");

Console.WriteLine($"p1 GetHashCode: {p1.GetHashCode()}");
Console.WriteLine($"p2 GetHashCode: {p2.GetHashCode()}");
Console.WriteLine($"p1.Equals(p2): {p1.Equals(p2)}");

p1 = p2; // Reassigning reference
Console.WriteLine($"After p1 = p2, p1.Equals(p2): {p1.Equals(p2)}");
```

Answers & Explanation:

1. **GetHashCode():** They give different hash code values. This happens because standard C# classes use reference equality. They get hash codes based on their memory location, not their values inside.
2. **patient01.Equals(patient02):** Returns False. Normal classes compare memory addresses (references). Since p1 and p2 are two different objects created in memory, they are not equal.
3. **After assigning patient01 = patient02:** Now it returns True. Both variables now point to the exact same memory location, so reference comparison passes.

Q3. Positional Record PatientDto & Comparisons

```
// Positional record definition
public record PatientDto(int Id, string FullName, string PhoneNumber);

// Testing in Main:
PatientDto dto1 = new(101, "Ahmed Nofal", "01012345678");
PatientDto dto2 = new(101, "Ahmed Nofal", "01012345678");

Console.WriteLine($"dto1 GetHashCode: {dto1.GetHashCode()}");
Console.WriteLine($"dto2 GetHashCode: {dto2.GetHashCode()}");
Console.WriteLine($"dto1.Equals(dto2): {dto1.Equals(dto2)}");
```

Difference Explanation (Class vs Record):

In classes, equality is based on *Reference Equality* (checking if both variables point to the same location in memory). In Records, C# automatically overrides `Equals()` and `GetHashCode()` to use *Value Equality*. This means two record instances are considered equal if all their properties contain the exact same values, even if they live in different memory locations.

Q4. PatientMapper Class (Static & Private Constructor)

```
public static class PatientMapper
{
    // Private constructor prevents instantiating this utility class
    static PatientMapper() { }

    public static PatientDto MapFromModelToDto(Patient patient)
    {
        return new PatientDto(patient.Id, patient.FullName, patient.PhoneNumber);
    }
}

// Usage in Main:
Patient patient = new(101, "Ahmed Nofal", "01012345678", "Confidential");
PatientDto dto = PatientMapper.MapFromModelToDto(patient);
Console.WriteLine(dto);
```

Part 2: Singleton Pattern

Q5 & Q6. AppLogger Singleton Implementation & Verification

```
public class AppLogger
{
    private static AppLogger? _instance = null;

    private AppLogger() { } // Private constructor to prevent direct instantiation

    public static AppLogger GetLogger()
    {
        if (_instance == null)
        {
            _instance = new AppLogger();
        }
        return _instance;
    }

    public void Log(string message) => Console.WriteLine($"[LOG]: {message}");
}

// Calling GetLogger() 4 times in Main:
var l1 = AppLogger.GetLogger();
var l2 = AppLogger.GetLogger();
var l3 = AppLogger.GetLogger();
var l4 = AppLogger.GetLogger();

Console.WriteLine(l1.GetHashCode());
Console.WriteLine(l2.GetHashCode());
Console.WriteLine(l3.GetHashCode());
Console.WriteLine(l4.GetHashCode());
```

Observation & Explanation:

All four printed hash codes are **100% identical**. This confirms that the Singleton pattern is working correctly. The class creates the object only once on the first call to `GetLogger()` and stores it in `_instance`. Any future calls simply return that existing object reference.

Part 3: var & dynamic

Q7. Object Declarations & Explanation of var vs dynamic

```
// 1. Target-typed new
Patient pTyped = new(102, "Mona", "0111111111", "Stable");

// 2. Using var keyword
var pVar = new Patient(103, "Omar", "0122222222", "Stable");

// 3. Using dynamic keyword
dynamic pDynamic = new Patient(104, "Sarah", "0155555555", "Checkup");
Console.WriteLine(pDynamic);
```

Difference between var and dynamic:

- var is **statically typed at compile time**. The compiler looks at the right side of the assignment and figures out the type. Once compiled, the type is fixed and cannot change. You get full IntelliSense and compiler checking.
- dynamic is **dynamically resolved at run time**. The compiler turns off compile-time type checking for this variable. Errors (like calling a method that doesn't exist) will only appear when the application runs, not during compilation.

Part 4: Anonymous Types

Q8. Anonymous Objects Creation & Method Results

```
var doctor01 = new { Name = "Sara", Specialty = "Cardiology", ExperienceYears = 8, Salary = 25000 };
var doctor02 = new { Name = "Sara", Specialty = "Cardiology", ExperienceYears = 8, Salary = 25000 };

// 1. Print Name and Specialty
Console.WriteLine($"Doctor 1: {doctor01.Name}, Specialty: {doctor01.Specialty}");

// 2. Print GetHashCode()
Console.WriteLine($"d1 GetHashCode: {doctor01.GetHashCode()}");
Console.WriteLine($"d2 GetHashCode: {doctor02.GetHashCode()}");

// 3. Print GetType()
Console.WriteLine($"Type: {doctor01.GetType()}");

// 4. Print Equals()
Console.WriteLine($"Equals: {doctor01.Equals(doctor02)}");

// 5. Print ToString()
Console.WriteLine($"ToString: {doctor01.ToString()}");
```

Comparison of Equality Types:

- **Class Equality:** Default behavior is *Reference Equality* (returns true only if both variables point to the exact same memory reference).
- **Record Equality:** Uses compiler-generated *Value Equality* based on all public properties.
- **Anonymous Type Equality:** Also uses *Value Equality*. The C# compiler automatically overrides `Equals()` and `GetHashCode()` for anonymous types, comparing all properties one by one. Therefore, `doctor01.Equals(doctor02)` evaluates to `True`.

Part 5: Extension Methods

Q9 & Q10. TextHelper Extension Methods & Usage

```
public static class TextHelper
{
    // Bonus: Normal static method example:
    // public static bool IsShorterThanNormal(string value, int length) => value.Length <
    length;

    // Extension method 1: IsShorterThan
    public static bool IsShorterThan(this string value, int length)
    {
        if (value == null) return false;
        return value.Length < length;
    }

    // Extension method 2: Repeat
    public static string Repeat(this string value, int times)
    {
        if (string.IsNullOrEmpty(value) || times <= 0) return string.Empty;
        var sb = new System.Text.StringBuilder();
        for (int i = 0; i < times; i++)
        {
            sb.Append(value);
        }
        return sb.ToString();
    }
}

// Usage in Main (Q10):
bool isShorter = "Stethoscope".IsShorterThan(5);
string repeated = "Ab".Repeat(4);

Console.WriteLine($"\"Stethoscope\".IsShorterThan(5): {isShorter}"); // False
Console.WriteLine($"\"Ab\".Repeat(4): {repeated}"); // AbAbAbAb
```

Part 6: Unit Testing (xUnit)

Q11. FineCalculator Class Implementation

```
public class FineCalculator
{
    public int Add(int a, int b) => a + b;
    public int Subtract(int a, int b) => a - b;
    public int Multiply(int a, int b) => a * b;

    public int Divide(int a, int b)
    {
        if (b == 0)
            throw new DivideByZeroException("Cannot divide by zero.");
        return a / b;
    }
}
```

Q12. xUnit Unit Test Implementation (FineCalculatorTests)

```
using Xunit;

public class FineCalculatorTests
{
    private readonly FineCalculator _calculator;

    public FineCalculatorTests()
    {
        // Arrange: setup instance in constructor
        _calculator = new FineCalculator();
    }

    [Fact]
    public void Add_ShouldReturnCorrectSum()
    {
        // Act
        int result = _calculator.Add(10, 5);
        // Assert
        Assert.Equal(15, result);
    }

    [Fact]
    public void Add_ShouldReturnNotEqual()
    {
        int result = _calculator.Add(10, 5);
        Assert.NotEqual(20, result);
    }

    [Fact]
    public void Subtract_ShouldReturnCorrectDifference()
    {
        int result = _calculator.Subtract(10, 4);
        Assert.Equal(6, result);
    }

    [Theory]
    [InlineData(3, 4, 12)]
    [InlineData(-2, 5, -10)] // Negative number case
    [InlineData(0, 10, 0)]
    public void Multiply_ShouldReturnCorrectResult(int a, int b, int expected)
    {
        int result = _calculator.Multiply(a, b);
        Assert.Equal(expected, result);
    }

    [Theory]
    [InlineData(3, 4, 99)]
    [InlineData(-2, 5, 100)]
    [InlineData(1, 1, 5)]
    public void Multiply_ShouldReturnNotEqual(int a, int b, int unexpected)
    {
        int result = _calculator.Multiply(a, b);
        Assert.NotEqual(unexpected, result);
    }
}
```

```
[Fact]
public void Divide_ByZero_ShouldThrowException()
{
    Assert.Throws<DivideByZeroException>(() => _calculator.Divide(10, 0));
}
}
```

Submission Verification Checklist

- Solution folder structure contains Console Project & xUnit Test Project.
- All Main questions are thoroughly answered with clear code and B1-level comments.
- All 6 required xUnit tests follow AAA structure and pass smoothly.
- Clean, readable C# formatting matching modern .NET guidelines.